ATRIX DIGITAL · ENGINEERING

The Atrix Harness

A complete guide for developers — what it is, how to set it up, and how to work with it day to day.

REPOSITORY	AUDIENCE	SET-UP TIME	VERSION
github.com/atrixdigital/atrix-harness	Every engineer at Atrix	About ten minutes	Living document

What this is

One repository that every Atrix engineer starts their AI agent from — carrying our rules, our conventions, our skills, and tools that let an agent understand a codebase without reading it file by file.

It works with whichever agent you use. Claude Code, Codex, Cursor and Gemini all read the same content, packaged differently for each. Launch through Orca and it still applies.

Why it exists

The scaffolding around a model matters at least as much as the model. Published measurements put the spread at **23.8 percentage points on identical tasks** from the harness alone, and the gains hold across every model tested. Separately, a pre-indexed code graph cuts roughly 59% of tokens and 70% of tool calls, because an agent stops rediscovering structure it could have looked up.

The second reason matters more over time. **It compounds.** When something costs you an hour, that hour is captured, reviewed, and turned into a rule everyone gets — instead of the next person losing the same hour.

What it is not

It is not a replacement for your agent, and it does not run the loop. Claude Code, Codex and Orca own that. The harness is the layer above: what the agent knows, what tools it has, and what stops it doing something stupid.

Ten minutes, once.

Clone the harness

This becomes your workspace. You launch your agent here, and every project lives inside it.

```
git clone https://github.com/atrixdigital/atrix-harness.git
cd atrix-harness
bun install
bun run atrix build
```

Put it on your PATH

```
# add to ~/.zshrc
export ATRIX_HOME="$HOME/path/to/atrix-harness"
alias atrix="bun run $ATRIX_HOME/packages/cli/src/index.ts"
```

> **ATRIX_HOME is not optional.** The graph server resolves it when your agent launches. Without it the tools fail with a path that looks malformed rather than a clear error.

Wire it into the workspace

```
atrix init
```

Writes the graph server config and index settings, once. It merges into an existing `.mcp.json` rather than replacing it, and refuses to touch one it cannot parse.

Install the plugin

Claude Code — from the workspace root:

```
/plugin marketplace add ./adapters/claude
/plugin install atrix-core@atrix-harness
/plugin install atrix-skills@atrix-harness
/plugin install atrix-graphs@atrix-harness
```

Codex or Gemini — point the agent at `adapters/codex/AGENTS.md` or `adapters/gemini/GEMINI.md`. Cursor — copy `adapters/cursor/.cursor/` into place. The graph server config is at `adapters/*/mcp.json`.

Check it actually works

```
atrix verify --live
```

```
✓ the marketplace is loadable
✓ components are discovered — 12 agents, 3 hook events
✓ the rules reach the model — retry (×2) → patch (×2) → replan, then stop
✓ the safety guard intercepts — canary survived, the guard asked
✓ our skills are discoverable — YES
```

> These launch real sessions and ask questions only answerable if each path works. They exist because every offline check we had was green at a time when the rules reached no Claude session at all. Structure passing is not the same as the model seeing it.

Clone it, then ask your agent. That is the whole procedure.

```
git clone git@github.com:atrixdigital/playo-web.git projects/playo-web
```

Then, in your agent:

> *"I've cloned playo-web into projects — set it up."*

The agent scaffolds it, reads the repository to work out its stack and its **real** command names, indexes it, audits its configuration, and records anything it worked out along the way. It usually offers before you ask — an un-onboarded project is flagged at session start.

Why not just run a command

`atrix init` writes the template. The valuable half needs someone to read the repo: `type-check` and `typecheck` are both common and only one exists in any given repo. A manual with the wrong command is worse than one with a blank, because somebody will run it.

What the project gets

FILE	HOLDS	LIVES IN
`AGENTS.md`	Stack, commands, conventions, gotchas	the project's own repo
`UNDERSTANDINGS.md`	How the system actually works, and why	the project's own repo

Both are committed to *that* repository, so they travel with the code and reach whoever clones it — with or without the harness.

> **Projects are independent repos and are gitignored here.** They keep their own remotes and history. Client code never merges into the shared harness.

The layout

```
atrix-harness/          ← launch your agent here
├── AGENTS.md           the manual every agent reads
├── core/               rules, skills, roles      shared
├── learning/           incidents                 shared
├── .atrix/             index, traces             yours, gitignored
└── projects/           gitignored
    ├── playo-web/      its own git remote
    └── ezrov/          its own git remote
```

At session start

A hook speaks only when something changes what you would do. Most sessions it says nothing. When it does:

```
Atrix harness — things worth knowing before you start:
- DATABASE_URL, SUPABASE_SERVICE_ROLE_KEY are defined in more than one env
  file with different values. Whichever file the running tool loads decides
  which system it talks to — confirm the target before any command that writes.
```

That is a real finding from a live repo. It fires *before* you run a migration, not after.

The rules

Eighteen rules load in every session — how code should fail, what needs confirmation, how to report honestly, git discipline, TypeScript conventions. Skills load only when relevant, so fourteen skills cost almost nothing until one applies.

Two hooks on every tool call

GUARD	WHAT IT DOES
Safety	Escalates destructive and outward-facing commands to you — `rm -rf`, force pushes, `DROP`, migrations, deploys, anything naming *prod*
Loop	Detects an agent repeating itself with no progress, and escalates before it burns your afternoon

```
call 3 → "You have made the same call 3 times with the same result…"
call 8 → "Bounded recovery says stop here… the next identical call goes to you."
call 9 → asks you
```

> Both **ask**; neither blocks silently. A guard that stops legitimate work gets switched off, and then it catches nothing.

Seven tools your agent can call, so it stops reading twenty files to answer one question.

TOOL	ANSWERS
`atrix_search`	Where is this symbol? Scoped to your project; can span every repo
`atrix_context`	Everything about one symbol in a single call
`atrix_callers`	What references this — before you change anything shared
`atrix_callees`	What this depends on
`atrix_impact`	Blast radius of a change, by distance
`atrix_env`	Which variables are read, where defined, where they disagree
`atrix_recall`	Why is it like this? What broke last time?

Cross-project search

Searches default to the project you are in. Ask for the whole workspace and the question becomes *has anyone already solved this*:

```
atrix_search createBooking --all-projects
→ playo-web/src/booking.ts:42
  ezrov/apps/api/src/booking.ts:88   ← the pattern already exists
```

Asking why

```
atrix recall "why do we commit generated adapters"
```

Searches recorded incidents, architecture understandings and handoffs — the reasoning the code does not state. The code graph tells you what calls what; this tells you why the retry lives in the consumer rather than the producer.

Checking config before it bites

```
atrix env
```

Reports variables read but never defined, and — the dangerous one — the same variable defined in two files with different values. **Values are never printed.** Definitions are compared by hash, so it says two differ without putting a credential in your terminal.

> **Run this before any migration.** The recurring failure across our repos is a tool loading `.env` while the app runs on `.env.local`, pointed at two different databases, both reporting success.

Keeping the index current

```
atrix index               # the project you are in
atrix index --all         # the whole workspace
```

Roughly a second for a small project, ten for a thousand files. The index is yours and gitignored; nothing leaves your machine.

This is the part that makes the system get better instead of going stale.

```
/learn                              # capture what bit you
atrix distill <incident>            # turn it into a proposed change
# open a PR — a human reviews it
atrix sync                          # everyone else gets it
```

`atrix sync` prints what the harness learned since your last pull, because an update nobody reads is an update nobody applies.

The most likely outcome is not a rule

Before proposing one, the question is always: **could a machine catch this instead?** A CI check, a hook, a type, a test. If it can, that beats a rule — it costs nothing at runtime and cannot be forgotten.

Three of our first five incidents resolved to a check rather than a rule. That is the loop working. A rule library grows by refusing things.

You do not have to remember

A hook records a redacted shape of every tool result. `atrix observe` surfaces failures that keep recurring, and the session-start hook flags the worst one. The person who has hit the same failure eleven times stopped noticing at the fourth.

> **Your traces are private.** They record the tool name, the program a shell call invoked — `psql`, never the query — and an error signature with paths, hostnames, numbers and quoted strings stripped. Gitignored regardless. Only reviewed incidents are shared.

Recording what you worked out

Different from an incident. An incident is what went wrong; an understanding is how something works. When you spend real time tracing a mechanism, put it in that

project's `UNDERSTANDINGS.md` with a date, a confidence level, and where it came from.

Dead ends count. A confirmed dead end is worth as much as a discovery, and it is lost completely if unwritten.

SHARED & COMMITTED	YOURS & GITIGNORED
`core/` — rules, skills, roles	`.atrix/graph.db` — you build your own
`learning/` — incidents merge via git	`.atrix/**/trace.jsonl`
each project's `UNDERSTANDINGS.md`, in its own repo	loop-guard state

Only distilled knowledge is shared. Your failures stay yours; a reviewed incident reaches everyone.

Staying current

```
atrix sync
```

Fast-forwards the harness, rebuilds the adapters, and prints the new entries. It refuses a dirty checkout — stashing your in-progress work to fetch someone else's is not a trade this tool makes.

`atrix status` tells you when you have fallen behind:

```
✗ harness up to date — 3 harness commit(s) behind — run `atrix sync`
```

Adding something for everyone

Read `CONTRIBUTING.md`. The short version, and the mistake worth avoiding:

WHERE	WHEN
The harness	True of *any* Atrix repo
That repo's `AGENTS.md`	True of one repo — its stack, its commands
That repo's `UNDERSTANDINGS.md`	Describes how it *works* rather than what to do

> **Rule or skill?** Rules load every session forever; skills load when relevant. Measured on this repo: one rule cost **625 tokens in every session**. Three skills of substantially more content cost **zero**. If it does not apply to every task, it is a skill.

7 · COMMAND REFERENCE

COMMAND	WHAT IT DOES
`atrix status`	Everything at a glance
`atrix doctor`	Is it wired up correctly
`atrix verify --live`	Prove it works against a real agent
`atrix init`	Set up the workspace, or a project when run inside one
`atrix index [--all]`	Build the code graph
`atrix env [--all]`	Audit environment variables
`atrix recall "<question>"`	Ask why something is the way it is
`atrix observe`	Failures that keep recurring
`atrix learn "<what bit you>"`	Capture an incident
`atrix distill [id]`	Turn one into a reviewable change
`atrix sync`	Pull the harness and rebuild
`atrix build`	Regenerate adapters after editing `core/`
`atrix lint`	Check skills against the authoring rules
`atrix eval`	Which layers have been measured

SYMPTOM	CAUSE AND FIX
Graph tools missing or failing	`ATRIX_HOME` not exported. Set it and restart the agent.
Search returns nothing	Not indexed. `atrix index`.
Results from the wrong project	`cd` into the project, or set `ATRIX_PROJECT`.
Impact looks stale	It is as current as the index. `atrix status` shows its age.
Agent does not know a rule	`atrix verify --live`. If the rules check fails, the delivery path is broken.
Guard did not fire	`atrix verify --live` tests it with a canary file.
Something worked yesterday	`atrix sync`, then `atrix doctor`.

Start here

```
atrix status     # the whole picture
atrix doctor     # nine checks, exits non-zero on a real problem
```

Being honest about what is proven

Three of thirty-one layers have an eval. The rules are researched and reasoned, and most have not been measured against agent behaviour. They are our best current judgement, not established fact — and each carries the incident or research that produced it, so you can judge for yourself.

If a rule gets in your way, that is a finding. Say so. Every rule also declares what would make it obsolete, and pruning is as much a contribution as adding.

> WHERE TO GO
>
> README.md — install and commands · CONTRIBUTING.md — adding to it, and what not to add
> docs/ARCHITECTURE.md — how it fits together · docs/RESEARCH.md — why it is shaped this way, with sources
> #core — ask. If the guide did not answer it, the guide is wrong and that is worth knowing.